

BOOK 1 — JAVA FUNDAMENTALS, METHODS & CONTROL STATEMENTS

FRESHER / ENTRY-LEVEL JAVA FULL STACK TEXTBOOK SERIES

TABLE OF CONTENTS

1. CHAPTER 1 — JAVA FUNDAMENTALS

- 1.1 Java Definition & Introduction
- 1.2 Features / Characteristics of Java
- 1.3 General-Purpose Language
- 1.4 Write Once, Run Anywhere (WORA)
- 1.5 Java vs C vs C++
- 1.6 History of Java
- 1.7 James Gosling, Sun Microsystems & Green Project
- 1.8 Oak → Java
- 1.9 JDK 1.0
- 1.10 Source Code, Bytecode & Machine Code
- 1.11 .java → Compiler → .class → JVM → Execution
- 1.12 JVM, JRE & JDK
- 1.13 Java Architecture
- 1.14 Compiler, Class Loader & Bytecode Verifier
- 1.15 Java Libraries
- 1.16 Interpreter & JIT Compiler
- 1.17 Java Memory Architecture — Basic
- 1.18 Portability
- 1.19 Basic Java Program Structure
- 1.20 package, class, main()
- 1.21 System.out.println() / print()

- 1.22 Escape Sequences
- 1.23 String Concatenation
- 1.24 Output Prediction
- 1.25 Basic Calculator
- 1.26 Operators
- 1.27 Arithmetic Operators
- 1.28 Relational Operators
- 1.29 Logical Operators
- 1.30 Assignment Operators
- 1.31 Unary Operators
- 1.32 Increment / Decrement
- 1.33 Ternary Operator
- 1.34 Bitwise Operators — Basic
- 1.35 Operator Precedence & Associativity
- Chapter 1 Assessment & Practice

2. CHAPTER 2 — DATA TYPES, VARIABLES & TYPE CASTING

- 2.1 Data Types
- 2.2 Primitive Data Types
- 2.3 All 8 Primitive Data Types
- 2.4 byte, short, int, long
- 2.5 float, double
- 2.6 char
- 2.7 boolean
- 2.8 Range of Primitive Data Types
- 2.9 Unicode
- 2.10 Reference Data Types
- 2.11 Class, Array & Enum as Reference Types
- 2.12 Variables
- 2.13 Local Variables
- 2.14 Instance Variables
- 2.15 Static Variables
- 2.16 Implicit / Widening Casting
- 2.17 Explicit / Narrowing Casting
- 2.18 Data Loss During Casting

- 2.19 float Suffix
- 2.20 Type Casting Programs
- 2.21 Hard Coding
- 2.22 Scanner & User Input
- 2.23 Even / Odd Program
- Chapter 2 Assessment & Practice

3. CHAPTER 3 — METHODS

- 3.1 Method Definition
- 3.2 Need for Methods
- 3.3 Predefined & User-defined Methods
- 3.4 Method Syntax
- 3.5 Four Types of Methods
- 3.6 Method Calling
- 3.7 Stack Frame
- 3.8 Method Memory
- 3.9 Local Variable Memory
- 3.10 Method Overloading
- 3.11 Rules of Method Overloading
- 3.12 Overloading main()
- 3.13 Calculator Using Methods
- 3.14 Largest of Three Numbers
 - 3.14.1 Largest of Two Numbers Using Method
 - 3.14.2 Smallest of Three Numbers Using Method
 - 3.14.3 Even / Odd Using Method
 - 3.14.4 Prime Number Using Method
- 3.15 Varargs
 - 3.15.1 Sum of Variable Number of Arguments
 - 3.15.2 Find Maximum Using Varargs
- 3.16 Recursion
 - 3.16.1 Factorial Using Recursion
 - 3.16.2 Fibonacci Using Recursion
 - 3.16.3 Reverse Number Using Recursion
- Chapter 3 Assessment & Practice

4. CHAPTER 4 — CONTROL STATEMENTS & LOOPS

- 4.1 Control Statements
- 4.2 if
- 4.3 if-else
- 4.4 else-if
- 4.5 if-else-if Ladder
- 4.6 Nested if
- 4.7 switch
- 4.8 for Loop
- 4.9 while Loop
- 4.10 do-while Loop
- 4.11 Entry-Controlled Loop
- 4.12 Exit-Controlled Loop
- 4.13 break
- 4.14 continue
- 4.15 return
- 4.16 Common Conditional & Loop Coding Problems
- 4.16.1 Even or Odd
- 4.16.2 Positive, Negative or Zero
- 4.16.3 Largest of Two Numbers
- 4.16.4 Largest of Three Numbers
- 4.16.5 Leap Year
- 4.16.6 Prime Number
- 4.16.7 Prime Numbers in a Range
- 4.16.8 Multiplication Table
- 4.16.9 Factorial
- 4.16.10 Fibonacci Series
- 4.16.11 Reverse Number
- 4.16.12 Palindrome Number
- 4.16.13 Sum of Digits
- 4.16.14 Count Digits
- 4.16.15 Armstrong Number
- 4.16.16 Perfect Number
- 4.16.17 Strong Number
- 4.16.18 Number Patterns

- 4.16.19 Star Patterns
- 4.16.20 Pyramid / Triangle Patterns
- Chapter 4 Assessment & Practice

5. END OF BOOK 1 — COMPREHENSIVE REVISION & INTERVIEW PACK

- 1. Complete Book 1 Revision Sheet
- 2. Important Fresher Interview Questions
- 3. Important Output-Prediction Questions
- 4. Important Coding Problems
- 5. Debugging Practice
- 6. Concept Comparison Questions
- 7. Fresher-Level Mock Interview
- 8. Final Coding Practice Set
- 9. Complete Book 1 Syllabus Coverage Checklist

CHAPTER 1 — JAVA FUNDAMENTALS

1.1 JAVA DEFINITION & INTRODUCTION

- **Topic Name:** Java Definition & Introduction
- **Definition:** Java is a high-level, class-based, object-oriented, general-purpose programming language designed to have as few implementation dependencies as possible.
- **Why Needed:** Traditional languages like C and C++ required platform-specific compilation. Java was created to enable software developers to write code once and run it across any operating system without modification.
- **Simple Explanation:** Imagine writing a instruction manual in a universal language that can be instantly translated and read by people speaking English, Spanish, or Japanese. That is Java for computers.
- **Real-World Analogy:** Standardized plug sockets and adapters across electrical appliances. You build one toaster, and with a standard adapter, it runs in any country's outlet.
- **Technical Explanation:** Java code is compiled into platform-independent intermediate code called **Bytecode** (.class file). The **Java Virtual Machine (JVM)** executes this bytecode by converting it to target machine code at runtime.
- **Syntax:**

```
`java  
  
public class HelloWorld {  
  
public static void main(String[] args) {
```

```
System.out.println("Hello, World!");  
}  
}  
`
```

- **Java Example:**

```
`java  
// HelloWorld.java - Basic introductory program in Java  
  
public class HelloWorld {  
    // Main entry point of any standalone Java application  
  
    public static void main(String[] args) {  
        // System.out.println prints message to console followed by a new line  
        System.out.println("Welcome to Java Full Stack Development!");  
    }  
}  
`
```

- **Output:**

```
`text  
  
Welcome to Java Full Stack Development!  
`
```

- **Program Explanation:** The class `HelloWorld` contains the `main` method. When executed, JVM looks for `main` and runs the statement inside it, outputting text to the console.
- **Execution Flow:** JVM starts -> Loads `HelloWorld.class` -> Invokes `main()` -> Executes `System.out.println()` -> Outputs text -> Method exits -> Program terminates.
- **Important Rules:** File name must match the public class name (`HelloWorld.java`). Java is strictly case-sensitive.
- **Common Mistakes:** Capitalization errors (`main` vs `Main`, `system` vs `System`), forgetting the semicolon `;`.
- **Common Interview Confusion:** Freshers often confuse Java with JavaScript. Java is a compiled/interpreted class-based OOP language; JavaScript is a dynamic scripting language for web browsers.
- **Important Interview Points:** Java was created by James Gosling at Sun Microsystems in 1995. Java is now owned by Oracle Corporation.
- **Memory/Revision Points:** `.java` -> Compiler (`javac`) -> `.class` (Bytecode) -> JVM -> Native Output.

1.2 FEATURES / CHARACTERISTICS OF JAVA

- **Topic Name:** Features / Characteristics of Java (Java Buzzwords)
- **Definition:** Primary attributes that define Java's capability, architecture, and popularity.

- **Why Needed:** Explains why enterprise systems prefer Java over C, C++, or Python.
- **Simple Explanation:** The "Java Buzzwords" are the 12 signature characteristics specified by Sun Microsystems.
- **Technical Explanation:**

1. **Simple:** Removes complex C++ features like explicit pointers, operator overloading, multiple inheritance through classes.
2. **Object-Oriented:** Everything (except primitives) is treated as an object.
3. **Platform Independent / Portable:** Write Once, Run Anywhere (WORA).
4. **Robust:** Strong memory management, automatic garbage collection, explicit exception handling.
5. **Secure:** Runs inside JVM sandbox, no explicit pointer arithmetic.
6. **Architecture-Neutral:** Fixed data type sizes regardless of 32-bit or 64-bit OS.
7. **Multithreaded:** Native support for concurrent execution.
8. **High Performance:** Just-In-Time (JIT) compiler speeds up bytecode execution.
9. **Distributed:** Native support for networking (RMI, HTTP, Sockets).
10. **Dynamic:** Dynamically loads classes on demand at runtime.

- **Important Rules:** Memory is managed automatically via Garbage Collector (GC).

- **Interview Questions:**

- *Q:* Why is Java secure? *A:* No explicit pointers and code runs inside JVM sandbox with Bytecode Verifier.

1.3 GENERAL-PURPOSE LANGUAGE

- **Topic Name:** General-Purpose Language
 - **Definition:** A language designed for building applications in a wide variety of domains rather than domain-specific tasks.
 - **Why Needed:** Developers don't need different languages for web backend, mobile apps, desktop software, and cloud microservices.
 - **Practical Usage:** Enterprise web apps (Spring Boot), Android apps, Big Data (Hadoop), Banking systems.
-

1.4 WRITE ONCE, RUN ANYWHERE (WORA)

- **Topic Name:** Write Once, Run Anywhere (WORA)
- **Definition:** Java's core philosophy where code compiled on one operating system can run on any other OS with a JVM.
- **Real-World Analogy:** PDF documents — created on Mac, viewable identically on Windows, Linux, Android.

- **Technical Explanation:** javac compiles .java into bytecode (.class). Bytecode is OS-neutral. Each OS has its specific JVM implementation that translates neutral bytecode into local machine instructions.
 - **Interview Confusion:** Is JVM platform-independent? **No.** JVM is platform-dependent (different for Windows, Linux, Mac). Bytecode is platform-independent.
-

1.5 JAVA VS C VS C++

- **Topic Name:** Java vs C vs C++

- **Comparison Table:**

Feature	C	C++	Java
Paradigm	Procedural	Object-Oriented & Procedural	Pure/Strict Object-Oriented
Memory Management	Manual (malloc/free)	Manual (new/delete)	Automatic (Garbage Collector)
Pointers	Explicit Pointers	Explicit Pointers	No Explicit Pointers
Platform Dependency	Platform Dependent	Platform Dependent	Platform Independent (WORA)
Multiple Inheritance	N/A	Supported (Classes)	Supported via Interfaces only

1.6 HISTORY OF JAVA

- **Topic Name:** History of Java

- **Key Milestones:**

- **1991:** Started as the **Green Project** by James Gosling, Mike Sheridan, and Patrick Naughton.
 - **Oak:** Initially named Oak after an oak tree outside James Gosling's office.
 - **1995:** Renamed to **Java** (inspired by Java coffee from Indonesia) and publicly released by Sun Microsystems.
 - **1996:** JDK 1.0 officially launched.
 - **2010:** Oracle Corporation acquired Sun Microsystems.
-

1.7 JAMES GOSLING, SUN MICROSYSTEMS & GREEN PROJECT

- **Topic Name:** James Gosling, Sun Microsystems & Green Project

- **Explanation:** James Gosling is known as the "Father of Java". The Green Project aimed to develop a language for interactive television and consumer electronic devices.
-

1.8 OAK → JAVA

- **Topic Name:** Oak → Java
 - **Explanation:** Renamed because "Oak" was already a registered trademark of Oak Technology. The team chose "Java" during a brainstorming session over coffee.
-

1.9 JDK 1.0

- **Topic Name:** JDK 1.0
 - **Explanation:** Released in January 1996. Provided initial core libraries (`java.lang`, `java.io`, `java.util`, `java.net`, `java.awt`).
-

1.10 SOURCE CODE, BYTECODE & MACHINE CODE

- **Topic Name:** Source Code, Bytecode & Machine Code
 - **Definitions:**
 - **Source Code** (`.java`): Human-readable code written by developers.
 - **Bytecode** (`.class`): Intermediate, platform-independent machine instructions generated by `javac`.
 - **Machine Code:** Native binary instructions (0s and 1s) directly executable by target CPU.
-

1.11 .JAVA → COMPILER → .CLASS → JVM → EXECUTION

- **Topic Name:** Compilation and Execution Pipeline
- **Execution Flow Diagram:**

`text

[Developer Writes Code] -> Hello.java

|

(javac Compiler)

```
v
[Bytecode Generated] -> Hello.class
|
(JVM Engine)
/|\
ClassLoader Verifier JIT/Interpreter
|
v
[Native CPU Execution] -> Output
`
```

1.12 JVM, JRE & JDK

- **Topic Name:** JVM vs JRE vs JDK
- **Relationship:** JDK = JRE + Development Tools | JRE = JVM + Library Classes
- **Definitions:**

- **JVM (Java Virtual Machine):** Abstract machine that executes Java bytecode.
 - **JRE (Java Runtime Environment):** Environment providing JVM + libraries required to run Java apps.
 - **JDK (Java Development Kit):** Complete software development suite providing JRE + compiler (javac), debugger, archiver (jar).
-

1.13 JAVA ARCHITECTURE

- **Topic Name:** Java Architecture
 - **Technical Explanation:** Architectural design separating code compilation from hardware execution. Bytecode acts as the universal bridge between high-level logic and hardware architecture.
-

1.14 COMPILER, CLASS LOADER & BYTECODE VERIFIER

- **Topic Name:** Compiler, Class Loader & Bytecode Verifier
 - **Components:**
- javac **Compiler:** Translates Java syntax into bytecode.

- **Class Loader:** Subsystem of JVM used to load class files into memory (Bootstrap, Extension, Application class loaders).
 - **Bytecode Verifier:** Security component that checks bytecode for illegal code, memory violations, or stack overflow risks before execution.
-

1.15 JAVA LIBRARIES

- **Topic Name:** Java Libraries (Standard API)
 - **Explanation:** Pre-compiled collections of classes and packages provided with the JDK (e.g., `java.lang`, `java.util`, `java.io`, `java.math`).
-

1.16 INTERPRETER & JIT COMPILER

- **Topic Name:** Interpreter & JIT (Just-In-Time) Compiler
 - **Explanation:**
- **Interpreter:** Reads bytecode line-by-line and executes it. Faster to start, slower for repeated code.
 - **JIT Compiler:** Compiles frequently executed bytecode ("hotspots") into native machine code at runtime. Significantly improves overall performance.
 - **Hybrid Engine:** JVM uses both Interpreter (for immediate execution) and JIT Compiler (for performance optimization).
-

1.17 JAVA MEMORY ARCHITECTURE — BASIC

- **Topic Name:** Java Memory Architecture — Basic
 - **Key Memory Areas:**
- **Class/Method Area:** Stores class structure, field metadata, method code, static variables.
 - **Heap Memory:** Stores all instantiated objects and instance variables.
 - **Stack Memory:** Stores method invocation frames, local variables, and primitive values.
 - **Program Counter (PC) Register:** Holds address of current JVM instruction being executed.
 - **Native Method Stack:** Holds instructions for C/C++ native library execution.
-

1.18 PORTABILITY

- **Topic Name:** Portability
 - **Explanation:** The ability to run identical bytecode on different CPU architectures (x86, ARM) and operating systems (Windows, Linux, macOS) without re-compilation.
-

1.19 BASIC JAVA PROGRAM STRUCTURE

- **Topic Name:** Basic Java Program Structure
- **Syntax Template:**

```
`java
// Package Declaration (Optional)
package com.fresher.book1;
# Import Statements (Optional)
import java.util.Scanner;
// Class Definition (Mandatory)
public class StructureDemo {
// Main Method Definition (Mandatory for executable applications)
public static void main(String[] args) {
// Statements / Logic
}
}
`
```

1.20 PACKAGE, CLASS, MAIN()

- **Topic Name:** package, class, main()
- **Explanations:**
 - package: Namespace that organizes related classes.
 - class: Blueprint/template defining state and behavior.
 - public static void main(String[] args):
 - public: Accessible from anywhere by JVM.
 - static: Can be called by JVM without creating an instance of the class.

- void: Returns no value.
 - main: Name of the entry point method recognized by JVM.
 - String[] args: Command-line arguments passed as an array of Strings.
-

1.21 SYSTEM.OUT.PRINTLN() / PRINT()

- **Topic Name:** System.out.println() vs System.out.print()

- **Technical Explanation:**

- System: Built-in final class in java.lang.
 - out: Static member field of System class of type PrintStream.
 - println(): Method of PrintStream that prints text and moves cursor to a new line.
 - print(): Method that prints text without adding a new line.
-

1.22 ESCAPE SEQUENCES

- **Topic Name:** Escape Sequences

- **Common Sequences:**

- \n: Newline
 - \t: Tab space
 - \\: Backslash
 - \": Double quote
 - \': Single quote
-

1.23 STRING CONCATENATION

- **Topic Name:** String Concatenation with + Operator

- **Rules:**

- If either operand is a String, + performs concatenation.
 - Evaluation is left-to-right.
 - Example: System.out.println("Sum: " + 10 + 20); -> Sum: 1020
 - Example: System.out.println(10 + 20 + " Sum"); -> 30 Sum
-

1.24 OUTPUT PREDICTION

- **Topic Name:** Output Prediction Exercises

- **Example 1:**

```
`java
public class Test {
public static void main(String[] args) {
System.out.println(10 + 20 + "Java" + 30 + 40);
}
}
`
```

- **Output:** 30Java3040
 - **Explanation:** Left to right: $10+20 = 30$. $30 + \text{"Java"} = \text{"30Java"}$. $\text{"30Java"} + 30 = \text{"30Java30"}$. $\text{"30Java30"} + 40 = \text{"30Java3040"}$.
-

1.25 BASIC CALCULATOR PROGRAM

- **Topic Name:** Basic Calculator Program (Console)

- **Java Program:**

```
`java
// BasicCalculator.java - Demonstrates arithmetic operations and formatting
public class BasicCalculator {
public static void main(String[] args) {
int num1 = 20;
int num2 = 10;
// Perform arithmetic operations
int add = num1 + num2;
int sub = num1 - num2;
int mul = num1 * num2;
int div = num1 / num2;
int mod = num1 % num2;
// Display results
```

```
System.out.println("Addition: " + num1 + " + " + num2 + " = " + add);
System.out.println("Subtraction: " + num1 + " - " + num2 + " = " + sub);
System.out.println("Multiplication: " + num1 + " * " + num2 + " = " + mul);
System.out.println("Division: " + num1 + " / " + num2 + " = " + div);
System.out.println("Modulus: " + num1 + " % " + num2 + " = " + mod);
}
}
```

- **Output:**

`text

Addition: 20 + 10 = 30

Subtraction: 20 - 10 = 10

Multiplication: 20 * 10 = 200

Division: 20 / 10 = 2

Modulus: 20 % 10 = 0

1.26 OPERATORS IN JAVA

- **Topic Name:** Operators Overview
- **Definition:** Special symbols used to perform operations on operands (variables and values).

1.27 ARITHMETIC OPERATORS

- **Operators:** + (Addition), - (Subtraction), * (Multiplication), / (Division), % (Modulus/Remainder).

1.28 RELATIONAL OPERATORS

- **Operators:** == (Equal to), != (Not equal to), > (Greater than), < (Less than), >= (Greater than or equal to), <= (Less than or equal to).
 - **Return Type:** Always returns boolean (true or false).
-

1.29 LOGICAL OPERATORS

- **Operators:**

- && (Logical AND) - Short-circuit: skips second operand if first is `false`.
 - || (Logical OR) - Short-circuit: skips second operand if first is `true`.
 - ! (Logical NOT) - Inverts boolean value.
-

1.30 ASSIGNMENT OPERATORS

- **Operators:** Simple (`=`), Compound (`+=`, `-=`, `*=`, `/=`, `%=`).
-

1.31 UNARY OPERATORS

- **Operators:** `+` (Unary plus), `-` (Unary minus), `++` (Increment), `--` (Decrement), `!` (Logical negation).
-

1.32 INCREMENT / DECREMENT

- **Topic Name:** Prefix vs Postfix Increment/Decrement
- **Prefix (`++i` / `--i`):** Increment/decrement value first, then use in expression.
- **Postfix (`i++` / `i--`):** Use current value in expression first, then increment/decrement.
- **Example Code:**

```
`java
public class IncDecDemo {
    public static void main(String[] args) {
        int a = 5;
        int b = ++a; // a becomes 6, b gets 6
        int c = a++; // c gets 6, a becomes 7
        System.out.println("a=" + a + ", b=" + b + ", c=" + c);
    }
}
```

- **Output:** a=7, b=6, c=6
-

1.33 TERNARY OPERATOR

- **Syntax:** variable = (condition) ? expression1 : expression2;

- **Example:**

```
`java
```

```
int x = 10, y = 20;
```

```
int max = (x > y) ? x : y; // max receives 20
```

1.34 BITWISE OPERATORS — BASIC

- **Operators:** & (Bitwise AND), | (Bitwise OR), ^ (Bitwise XOR), ~ (Bitwise Complement), << (Left Shift), >> (Right Shift), >>> (Unsigned Right Shift).
-

1.35 OPERATOR PRECEDENCE & ASSOCIATIVITY

- **Precedence Hierarchy (High to Low):**

1. Postfix (expr++, expr--)
2. Unary (++expr, --expr, +, -, !, ~)
3. Multiplicative (*, /, %)
4. Additive (+, -)
5. Shift (<<, >>, >>>)
6. Relational (<, >, <=, >=, instanceof)
7. Equality (==, !=)
8. Bitwise AND (&)
9. Bitwise XOR (^)
10. Bitwise OR (|)
11. Logical AND (&&)

- 12. Logical OR (| |)
 - 13. Ternary (?:)
 - 14. Assignment (=, +=, -=, etc.)
-

CHAPTER 1 — ASSESSMENT & PRACTICE

Chapter Summary

Chapter 1 covered Java fundamentals, history, WORA architecture, JDK/JRE/JVM ecosystem, program structure, print formatting, and the complete operator set.

Interview Questions & Answers

1. What makes Java platform independent?

Answer: Java code compiles to intermediate bytecodes (.class), which run on any OS possessing a Java Virtual Machine (JVM).

2. Difference between JDK, JRE, and JVM?

Answer: JVM executes bytecode; JRE provides JVM + standard runtime libraries; JDK contains JRE + development tools (javac).

Output-Based Practice

• Predict Output:

```
`java
int a = 5;
System.out.println(a++ + ++a * 2);
`
```

• **Solution:** a++ evaluates to 5 (a becomes 6). ++a increments a to 7 and evaluates to 7. $5 + (7 * 2) = 5 + 14 = 19$.

Quick Revision Checklist

- [x] Know Java history, Green project, James Gosling
- [x] Understand WORA and Bytecode mechanism
- [x] Explain JDK vs JRE vs JVM
- [x] Master Operator Precedence and Prefix/Postfix increment

CHAPTER 2 — DATA TYPES, VARIABLES & TYPE CASTING

2.1 DATA TYPES IN JAVA

- **Topic Name:** Data Types Overview
 - **Definition:** A data type specifies the size, range, and type of values that a variable can store, along with the set of operations that can be performed on it.
 - **Why Needed:** Computers need to allocate specific memory addresses and byte sizes based on what is stored (e.g., an integer vs a text sentence).
-

2.2 PRIMITIVE DATA TYPES

- **Definition:** Built-in data types provided by Java that store actual values directly in stack memory.
 - **Characteristics:** Reserved keywords, predefined fixed size, no methods attached.
-

2.3 ALL 8 PRIMITIVE DATA TYPES

- **Summary Table:**

Data Type	Category	Size	Default Value	Range
---	---	---	---	---
byte	Integer	1 byte (8 bits)	0	-128 to 127
short	Integer	2 bytes (16 bits)	0	-32,768 to 32,767
int	Integer	4 bytes (32 bits)	0	-2^{31} to $2^{31} - 1$
long	Integer	8 bytes (64 bits)	0L	-2^{63} to $2^{63} - 1$
float	Floating point	4 bytes (32 bits)	0.0f	~6-7 decimal digits
double	Floating point	8 bytes (64 bits)	0.0d	~15-16 decimal digits
char	Character	2 bytes (16 bits)	\u0000	0 to 65,535 (Unicode)
boolean	Logical	JVM-dependent	false	true or false

2.4 BYTE, SHORT, INT, LONG

- **Deep Dive:**

- `byte`: Memory-efficient for small numbers in raw arrays.
 - `short`: 16-bit signed integer.
 - `int`: Default data type for integral numbers in Java.
 - `long`: Used for huge values (population, currency in cents). Requires `L` or `l` suffix when assigning literal outside `int` range (e.g., `10000000000L`).
-

2.5 FLOAT, DOUBLE

- **Deep Dive:**

- `float`: Single-precision 32-bit IEEE 754 floating point. Requires `f` or `F` suffix (e.g., `3.14f`).
 - `double`: Double-precision 64-bit IEEE 754 floating point. Default choice for decimal values.
-

2.6 CHAR DATA TYPE

- **Deep Dive:**

- Stores a single 16-bit Unicode character enclosed in single quotes (`'A'`).
 - Converts internally to numeric Unicode values (`'A'` = 65, `'a'` = 97, `'0'` = 48).
-

2.7 BOOLEAN DATA TYPE

- **Deep Dive:**

- Holds only `true` or `false`. Cannot be converted to `1` or `0` like in C/C++.
-

2.8 RANGE OF PRIMITIVE DATA TYPES

- **Formula:** For n-bit signed integer: $-2^{(n-1)}$ to $2^{(n-1)}-1$.
 - **Example:** For `byte` (n=8): -2^7 to 2^7-1 = -128 to 127.
-

2.9 UNICODE SYSTEM

- **Explanation:** Java uses 16-bit Unicode (\uXXXX) to support international scripts (Greek, Chinese, Devanagari, Arabic, Emoji). C uses 8-bit ASCII.
-

2.10 REFERENCE DATA TYPES

- **Definition:** Non-primitive data types that store the memory address (reference) of an object located in Heap memory.
 - **Default Value:** null.
-

2.11 CLASS, ARRAY & ENUM AS REFERENCE TYPES

- **Explanation:**
 - **Class:** User-defined reference type (e.g., String, Scanner, custom classes).
 - **Array:** Homogeneous collection object.
 - **Enum:** Type representing a fixed group of constants.
-

2.12 VARIABLES IN JAVA

- **Definition:** Named memory locations used to store data values during program execution.
 - **Syntax:** `dataType variableName = initialValue;`
-

2.13 LOCAL VARIABLES

- **Scope:** Declared inside a method, constructor, or block.
 - **Memory:** Stack memory.
 - **Rules:** Must be initialized before use. No default values.
-

2.14 INSTANCE VARIABLES

- **Scope:** Declared inside a class but outside methods.
 - **Memory:** Heap memory (inside the object instance).
 - **Rules:** Automatically initialized with default values (0, null, false).
-

2.15 STATIC VARIABLES

- **Scope:** Declared with `static` keyword inside a class.
 - **Memory:** Class area (shared across all instances of the class).
-

2.16 IMPLICIT / WIDENING TYPE CASTING

- **Definition:** Automatic conversion of a smaller data type to a larger data type by the compiler without data loss.
- **Flow:** `byte -> short -> int -> long -> float -> double`
- **Example:**

```
`java
int num = 100;
double d = num; // Automatic widening casting
`
```

2.17 EXPLICIT / NARROWING TYPE CASTING

- **Definition:** Manual conversion of a larger data type to a smaller data type using cast operator (`targetType`).
 - **Flow:** `double -> float -> long -> int -> short -> byte`
 - **Syntax:** `targetType var2 = (targetType) var1;`
-

2.18 DATA LOSS DURING CASTING

- **Explanation:** Narrowing casting strips higher-order bits, leading to overflow or truncation.
- **Example:**

```
`java
int val = 130;
byte b = (byte) val; // Output: -126 (due to byte range -128 to 127 overflow)
`
```

2.19 FLOAT SUFFIX REQUIREMENT

- **Rule:** Fractional literals are treated as double by default. Assigning 3.14 to a float variable causes a compilation error (Incompatible types: possible lossy conversion from double to float).
 - **Fix:** Append f or F: float price = 3.14f;.
-

2.20 TYPE CASTING PROGRAMS

- **Java Program:**

```
`java
// TypeCastingDemo.java - Demonstrates Widening and Narrowing Casting
public class TypeCastingDemo {
    public static void main(String[] args) {
        // Widening casting (Implicit)
        int intVal = 250;
        double doubleVal = intVal; // Automatically converted
        System.out.println("Implicit Casting (int to double): " + doubleVal);

        // Narrowing casting (Explicit)
        double price = 99.99;
        int roundedPrice = (int) price; // Fractional part dropped
        System.out.println("Explicit Casting (double to int): " + roundedPrice);

        // Overflow demonstration
        int largeNum = 130;
        byte byteNum = (byte) largeNum;
        System.out.println("Overflow Casting (130 to byte): " + byteNum);
    }
}
```

- **Output:**

```
`text
```

Implicit Casting (int to double): 250.0

Explicit Casting (double to int): 99

Overflow Casting (130 to byte): -126

2.21 HARD CODING

- **Definition:** Practice of embedding fixed values directly in source code instead of accepting input dynamically.
-

2.22 SCANNER & USER INPUT

- **Explanation:** Scanner class from `java.util` reads user input from `System.in`.

- **Common Scanner Methods:**

- `nextInt()`: Reads integer.

- `nextDouble()`: Reads double.

- `next()`: Reads single word.

- `nextLine()`: Reads entire line.

2.23 EVEN / ODD PROGRAM (WITH SCANNER)

- **Java Program:**

```
`java
```

```
// EvenOddUser.java - Accepts integer from user and checks even/odd
```

```
import java.util.Scanner;
```

```
public class EvenOddUser {
```

```
    public static void main(String[] args) {
```

```
        // Create Scanner object for standard console input
```

```
        Scanner scanner = new Scanner(System.in);
```

```
System.out.print("Enter an integer number: ");
int number = scanner.nextInt();
// Check even/odd using modulus operator
if (number % 2 == 0) {
    System.out.println(number + " is an EVEN number.");
} else {
    System.out.println(number + " is an ODD number.");
}
// Good practice: Close the scanner stream
scanner.close();
}
}
```

CHAPTER 2 — ASSESSMENT & PRACTICE

Chapter Summary

Chapter 2 explained Java data types (8 primitives vs reference types), variable scopes, type conversion mechanisms, overflow risks, and dynamic keyboard input handling via Scanner.

Interview Questions

1. Why does char take 2 bytes in Java?

Answer: Java uses Unicode encoding rather than ASCII to support international languages, which requires 16 bits (2 bytes).

2. What is the result of (byte) 128?

Answer: -128.

Quick Revision Checklist

- [x] Remember exact sizes and default values of all 8 primitives
- [x] Master Widening vs Narrowing casting
- [x] Know why `float f = 3.14;` fails without `f` suffix
- [x] Understand Local, Instance, and Static variable scope differences

CHAPTER 3 — METHODS

3.1 METHOD DEFINITION

- **Topic Name:** Method Definition
 - **Definition:** A method is a collection of statements grouped together to perform a specific task or operation.
 - **Why Needed:** Promotes code reusability, modularity, readability, and ease of debugging (DRY Principle — Don't Repeat Yourself).
-

3.2 NEED FOR METHODS

- **Explanation:** Without methods, duplicate logic is repeated across multiple places. Updating logic requires modifying every single occurrence, leading to bugs. Methods centralize logic into one reusable block.
-

3.3 PREDEFINED & USER-DEFINED METHODS

- **Predefined Methods:** Built into Java API packages (e.g., `Math.sqrt()`, `System.out.println()`, `String.length()`).
 - **User-Defined Methods:** Created by developers to execute custom application logic.
-

3.4 METHOD SYNTAX

- **Syntax Template:**

```
`java
accessModifier returnType methodName(parameterList) {
// Method body / Statements
return value; // Required if returnType is not void
}
`
```

- **Components:**

- **accessModifier:** Controls visibility (public, private, protected, default).
- **returnType:** Data type of returned value (int, double, String, or void).

- `methodName`: Valid identifier following camelCase convention.
 - `parameterList`: Inputs passed into the method separated by commas.
-

3.5 FOUR TYPES OF METHODS

1. No Arguments and No Return Value:

```
void displayMessage()
```

2. With Arguments and No Return Value:

```
void printSum(int a, int b)
```

3. No Arguments and With Return Value:

```
int getRandomNumber()
```

4. With Arguments and With Return Value:

```
int calculateSquare(int n)
```

3.6 METHOD CALLING & EXECUTION

- **Explanation:** Calling a method pauses current execution flow, pushes a new frame onto Stack Memory, passes arguments, executes statements, pops frame, and returns control to caller.
-

3.7 STACK FRAME & METHOD MEMORY

- **Technical Explanation:** Each method invocation creates a discrete **Stack Frame** in Call Stack memory containing:
 - Local variables
 - Operand stack
 - Frame data (return address)
-

3.8 LOCAL VARIABLE MEMORY IN METHODS

- **Rule:** Local variables declared inside a method are created when stack frame is pushed and destroyed when stack frame is popped upon method return.

3.10 METHOD OVERLOADING

- **Definition:** Defining multiple methods in the same class with the **same name** but **different parameter lists** (compile-time polymorphism).
-

3.11 RULES OF METHOD OVERLOADING

- **Allowed Variations:**

1. Different number of parameters.
2. Different data types of parameters.
3. Different sequence of parameter data types.

- **Rule:** Changing return type alone is NOT sufficient for method overloading (causes compilation error).
-

3.12 OVERLOADING MAIN() METHOD

- **Explanation:** Yes, main() method can be overloaded in Java! However, JVM always searches specifically for `public static void main(String[] args)` as application entry point.
-

3.13 CALCULATOR USING METHODS

- **Java Program:**

```
`java
// MethodCalculator.java - Clean calculator implementation using methods
public class MethodCalculator {
// Method for Addition
public static double add(double a, double b) {
return a + b;
}
// Method for Subtraction
public static double subtract(double a, double b) {
```

```
return a - b;
}

// Method for Multiplication
public static double multiply(double a, double b) {
return a * b;
}

// Method for Division
public static double divide(double a, double b) {
if (b == 0) {
System.out.println("Error: Cannot divide by zero!");
return Double.NaN;
}
return a / b;
}

public static void main(String[] args) {
double x = 50.0, y = 10.0;
System.out.println("Add: " + add(x, y));
System.out.println("Subtract: " + subtract(x, y));
System.out.println("Multiply: " + multiply(x, y));
System.out.println("Divide: " + divide(x, y));
}
}
```

3.14 METHOD-BASED CODING PROBLEMS

3.14.1 Largest of Two Numbers Using Method

```
public static int maxTwo(int a, int b) {
    return (a > b) ? a : b;
}
```

3.14.2 Smallest of Three Numbers Using Method

```
public static int minThree(int a, int b, int c) {  
    int min = a;  
    if (b < min) min = b;  
    if (c < min) min = c;  
    return min;  
}
```

3.14.3 Even / Odd Using Method

```
public static boolean isEven(int num) {  
    return num % 2 == 0;  
}
```

3.14.4 Prime Number Using Method

```
public static boolean isPrime(int num) {  
    if (num <= 1) return false;  
    for (int i = 2; i <= Math.sqrt(num); i++) {  
        if (num % i == 0) return false;  
    }  
    return true;  
}
```

3.14.5 Factorial Using Method

```
public static long factorial(int n) {  
    long fact = 1;  
    for (int i = 1; i <= n; i++) {  
        fact *= i;  
    }  
    return fact;  
}
```

3.14.6 Reverse Number Using Method

```
public static int reverseNumber(int n) {  
    int rev = 0;  
    while (n != 0) {  
        int rem = n % 10;  
        rev = rev * 10 + rem;  
        n /= 10;  
    }  
    return rev;  
}
```

3.14.7 Palindrome Number Using Method

```
public static boolean isPalindrome(int n) {  
    return n == reverseNumber(n);  
}
```

3.15 VARARGS (VARIABLE-LENGTH ARGUMENTS)

- **Syntax:** returnType methodName(dataType... variableName)
- **Rule:** Varargs parameter must be the **last parameter** in the method signature. Only one varargs parameter per method allowed.

3.15.1 Sum of Variable Number of Arguments

```
public static int sumVarargs(int... numbers) {  
    int total = 0;  
    for (int num : numbers) {  
        total += num;  
    }  
    return total;  
}
```

3.15.2 Find Maximum Using Varargs

```
public static int maxVarargs(int first, int... rest) {  
    int max = first;  
    for (int val : rest) {  
        if (val > max) max = val;  
    }  
    return max;  
}
```

3.16 RECURSION IN JAVA

- **Definition:** Mechanism where a method calls itself directly or indirectly to break down a problem into smaller sub-problems.

- **Mandatory Requirements:**

1. **Base Case:** Stopping condition that terminates recursion.
2. **Recursive Call:** Method calling itself with modified inputs approaching the base case.

- **Risk:** Missing base case causes `StackOverflowError`.

3.16.1 Factorial Using Recursion

```
public static long factRecursion(int n) {  
    if (n <= 1) return 1; // Base case  
    return n * factRecursion(n - 1); // Recursive call  
}
```

3.16.2 Fibonacci Series Using Recursion

```
public static int fibRecursion(int n) {  
    if (n <= 0) return 0;  
    if (n == 1) return 1;  
    return fibRecursion(n - 1) + fibRecursion(n - 2);  
}
```

3.16.3 Reverse Number Using Recursion

```
public static void printReverseRecursion(int n) {  
    if (n < 10) {  
        System.out.print(n);  
        return;  
    }  
    System.out.print(n % 10);  
    printReverseRecursion(n / 10);  
}
```

CHAPTER 3 — ASSESSMENT & PRACTICE

Chapter Summary

Chapter 3 explained method declaration, types of methods, stack frame lifecycle, method overloading rules, varargs syntax, and recursion implementation.

Quick Revision Checklist

- [x] Know method signature components
- [x] Understand Stack Frame allocation and call stack
- [x] Master Method Overloading rules (parameter difference, not return type)
- [x] Write recursive functions with proper base cases

CHAPTER 4 — CONTROL STATEMENTS & LOOPS

4.1 CONTROL STATEMENTS OVERVIEW

- **Definition:** Control statements alter the natural sequential execution flow of a program based on conditional logic or loops.
- **Categories:**

1. **Decision Making:** if, if-else, switch
 2. **Looping / Iteration:** for, while, do-while
 3. **Branching / Jump:** break, continue, return
-

4.2 IF STATEMENT

- **Syntax:**

```
`java
if (condition) {
// Code executes if condition is true
}
`
```

4.3 IF-ELSE STATEMENT

- **Syntax:**

```
`java
if (condition) {
// Code if true
} else {
// Code if false
}
`
```

4.4 ELSE-IF & 4.5 IF-ELSE-IF LADDER

- **Syntax:**

```
`java
if (condition1) {
// Logic 1
} else if (condition2) {
```

```
// Logic 2
} else {
// Default logic
}
`
```

4.6 NESTED IF

- **Explanation:** An `if` statement located inside another `if` block. Used for multi-level hierarchical conditions.
-

4.7 SWITCH STATEMENT

- **Syntax:**

```
`java
switch (expression) {
case value1:
// Statements
break;
case value2:
// Statements
break;
default:
// Default statements
}
`
```

- **Allowed Types:** `byte`, `short`, `char`, `int`, `String`, and `enum`.
-

4.8 FOR LOOP

- **Syntax:**

```
`java
```

```
for (initialization; condition; increment/decrement) {  
    // Loop body  
}
```

4.9 WHILE LOOP

- **Syntax:**

```
`java  
while (condition) {  
    // Loop body  
}
```

4.10 DO-WHILE LOOP

- **Syntax:**

```
`java  
do {  
    // Loop body executes at least once  
} while (condition);
```

4.11 ENTRY-CONTROLLED VS 4.12 EXIT-CONTROLLED LOOPS

- **Entry-Controlled** (for, while): Condition tested BEFORE executing body. If condition is false initially, body never runs.
 - **Exit-Controlled** (do-while): Condition tested AFTER executing body. Body always executes at least ONCE regardless of condition.
-

4.13 BREAK STATEMENT

- **Explanation:** Immediately terminates nearest enclosing loop or switch block.
-

4.14 CONTINUE STATEMENT

- **Explanation:** Skips remaining statements in current loop iteration and proceeds to next iteration.
-

4.15 RETURN STATEMENT

- **Explanation:** Exits current method immediately and passes control back to calling method, optionally returning a value.
-

4.16 CONDITIONAL & LOOP CODING PROBLEMS

4.16.1 Even or Odd

```
int n = 7;
if (n % 2 == 0) System.out.println("Even");
else System.out.println("Odd");
```

4.16.2 Positive, Negative or Zero

```
int num = -15;
if (num > 0) System.out.println("Positive");
else if (num < 0) System.out.println("Negative");
else System.out.println("Zero");
```

4.16.3 Largest of Two Numbers

```
int a = 25, b = 40;
int max = (a > b) ? a : b;
```

4.16.4 Largest of Three Numbers

```
int x = 10, y = 25, z = 15;
if (x >= y && x >= z) System.out.println("Largest: " + x);
else if (y >= x && y >= z) System.out.println("Largest: " + y);
else System.out.println("Largest: " + z);
```

4.16.5 Leap Year

```
int year = 2024;
boolean isLeap = (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
System.out.println(year + " is leap year: " + isLeap);
```

4.16.6 Prime Number

```
int n = 29;
boolean prime = n > 1;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) { prime = false; break; }
}
System.out.println(n + " is prime: " + prime);
```

4.16.7 Prime Numbers in a Range (1 to N)

```
int start = 10, end = 50;
for (int i = start; i <= end; i++) {
    if (i <= 1) continue;
    boolean isP = true;
    for (int j = 2; j * j <= i; j++) {
        if (i % j == 0) { isP = false; break; }
    }
    if (isP) System.out.print(i + " ");
}
}
```

4.16.8 Multiplication Table

```
int num = 5;
for (int i = 1; i <= 10; i++) {
    System.out.println(num + " x " + i + " = " + (num * i));
}
```

4.16.9 Factorial

```
int n = 5;
long fact = 1;
for (int i = 1; i <= n; i++) fact *= i;
System.out.println("Factorial of " + n + " = " + fact);
```

4.16.10 Fibonacci Series

```
int terms = 10;
int t1 = 0, t2 = 1;
System.out.print("Fibonacci: ");
for (int i = 1; i <= terms; i++) {
    System.out.print(t1 + " ");
    int sum = t1 + t2;
    t1 = t2;
    t2 = sum;
}
```

4.16.11 Reverse Number

```
int num = 1234;
int rev = 0;
while (num != 0) {
    rev = rev * 10 + (num % 10);
    num /= 10;
}
System.out.println("Reversed: " + rev);
```

4.16.12 Palindrome Number

```
int original = 121, temp = original, rev = 0;
while (temp != 0) {
    rev = rev * 10 + (temp % 10);
    temp /= 10;
}
System.out.println("Is Palindrome: " + (original == rev));
```

4.16.13 Sum of Digits

```
int n = 456, sum = 0;
while (n != 0) {
    sum += (n % 10);
    n /= 10;
}
System.out.println("Sum of digits: " + sum);
```

4.16.14 Count Digits

```
int n = 98765, count = 0;
while (n != 0) {
    count++;
    n /= 10;
}
System.out.println("Digit count: " + count);
```

4.16.15 Armstrong Number (e.g., $153 = 1^3 + 5^3 + 3^3$)

```
int num = 153, original = num, result = 0;
while (original != 0) {
    int remainder = original % 10;
    result += Math.pow(remainder, 3);
    original /= 10;
}
System.out.println("Is Armstrong: " + (result == num));
```

4.16.16 Perfect Number (Sum of proper divisors equals number, e.g., $6 = 1+2+3$)

```
int n = 28, sum = 0;
for (int i = 1; i <= n / 2; i++) {
    if (n % i == 0) sum += i;
}
System.out.println("Is Perfect Number: " + (sum == n));
```

4.16.17 Strong Number (Sum of factorials of digits equals number, e.g., $145 = 1! + 4! + 5!$)

```
int num = 145, temp = num, sum = 0;
while (temp != 0) {
    int rem = temp % 10;
    long f = 1;
    for (int i = 1; i <= rem; i++) f *= i;
    sum += f;
    temp /= 10;
}
System.out.println("Is Strong Number: " + (sum == num));
```

4.16.18 Number Patterns

```
// Right Angle Number Triangle
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j + " ");
    }
    System.out.println();
}
```

4.16.19 Star Patterns

```
// Right Angle Star Pattern
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

4.16.20 Pyramid / Triangle Patterns

```
// Full Star Pyramid
int rows = 5;
for (int i = 1; i <= rows; i++) {
    for (int j = i; j < rows; j++) {
        System.out.print(" ");
    }
    for (int k = 1; k <= (2 * i - 1); k++) {
        System.out.print("*");
    }
    System.out.println();
}
```

CHAPTER 4 — ASSESSMENT & PRACTICE

Chapter Summary

Chapter 4 covered decision-making statements (if, switch), loop mechanisms (for, while, do-while), jump statements (break, continue, return), and implemented 20 classic logical interview coding problems.

Quick Revision Checklist

- [x] Distinguish entry-controlled vs exit-controlled loops
- [x] Know switch allowed data types
- [x] Implement prime, fibonacci, armstrong, perfect, and strong number logic
- [x] Master nested loop star and number pattern logic

END OF BOOK 1 — COMPREHENSIVE REVISION & INTERVIEW PACK

1. COMPLETE BOOK 1 REVISION SHEET

- **Java History:** James Gosling, Green Project, Oak -> Java (1995), Sun Microsystems -> Oracle (2010).
 - **WORA Mechanism:** Source (.java) -> Compiler (javac) -> Bytecode (.class) -> JVM -> Native OS Execution.
 - **JDK vs JRE vs JVM:** JDK = JRE + Tools | JRE = JVM + Libraries | JVM executes bytecode.
 - **8 Primitives:** byte (1B), short (2B), int (4B), long (8B), float (4B), double (8B), char (2B Unicode), boolean (logical).
 - **Type Casting:** Widening (implicit, no loss) vs Narrowing (explicit, possible data loss).
 - **Variables:** Local (stack, no default), Instance (heap, default values), Static (class area, shared).
 - **Methods & Stack:** Methods run in Stack Frames. Overloading requires different parameters (same name).
 - **Recursion:** Function calling itself; requires base case to prevent StackOverflowError.
 - **Loops:** Entry-controlled (for, while) vs Exit-controlled (do-while).
-

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. Why is Java not 100% Object-Oriented?

Answer: Because it supports 8 primitive data types (int, char, etc.) which are not objects.

2. Can main() method be private or protected?

Answer: No, JVM looks for public static void main(String[] args) specifically.

3. Difference between System.out.println() and System.out.print()?

Answer: println() appends a newline character after printing, whereas print() leaves the cursor on the same line.

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

1. **Code:**

```
`java
int x = 10;
System.out.println(x++ + ++x);
`
```

Output: 22 (Explanation: $10 + 12 = 22$).

2. **Code:**

```
`java
System.out.println('A' + 'B');
`
```

Output: 131 (Explanation: Character arithmetic converts chars to ASCII/Unicode numbers: $65 + 66 = 131$).

4. IMPORTANT CODING PROBLEMS

1. Swap two numbers without using a third variable.
 2. Check if a number is Armstrong or Strong number.
 3. Print prime numbers between 1 and 100.
 4. Generate Fibonacci sequence using both iteration and recursion.
-

5. DEBUGGING PRACTICE

• **Buggy Code:**

```
`java
public class Test {
    public static void main(String[] args) {
        float f = 3.14; // Compilation Error!
        System.out.println(f);
    }
}
```

- **Fix:** Append f suffix: `float f = 3.14f;`.
-

6. CONCEPT COMPARISON QUESTIONS

- **Comparing while vs do-while:** while checks condition before body; do-while checks after body (executes at least once).
 - **Comparing == vs .equals():** == compares primitives by value or references by memory address; .equals() compares string/object contents.
-

7. FRESHER-LEVEL MOCK INTERVIEW

- **Interviewer:** "What happens if you remove static from public static void main?"
 - **Candidate Answer:** "The code compiles fine, but at execution time JVM throws `NoSuchMethodError: main method not static in class` because JVM calls `main()` directly using the class name without creating an object."
-

8. FINAL CODING PRACTICE SET

Solve the following in your IDE:

1. Print a Hollow Star Pyramid pattern.
 2. Write a recursive program to compute the power of a number (x^n).
 3. Write a program using Scanner that validates user age for voting eligibility.
-

9. COMPLETE BOOK 1 SYLLABUS COVERAGE CHECKLIST

- [x] 1. Core Java — Java Fundamentals (1.1 - 1.35) — 100% Covered
 - [x] 2. Data Types, Variables & Type Casting (2.1 - 2.23) — 100% Covered
 - [x] 3. Methods (3.1 - 3.16.3) — 100% Covered
 - [x] 4. Control Statements & Loops (4.1 - 4.16.20) — 100% Covered
-

BOOK 1 COMPLETE — READY FOR BOOK 2